

HTML + CSS

Building and Styling the Web

CSCI 39548 — Practical Web Development

Section Goals

- ▶ Understand what HTML is and how browsers read it
- ▶ Write well-structured HTML documents
- ▶ Style pages with CSS (inline, internal, external)
- ▶ Use Flexbox and Grid for layout
- ▶ Build responsive, polished pages with transitions

Section Items

- ▶ HTML Primitives
- ▶ Lists and Tables
- ▶ Semantic HTML
- ▶ Forms and Inputs
- ▶ CSS — Inline / Internal / External
- ▶ Selectors and the Box Model
- ▶ Typography, Units, and CSS Variables
- ▶ Flexbox
- ▶ Grid and Positioning
- ▶ Responsive Design and Transitions

Section Items

► HTML Primitives

- Lists and Tables
- Semantic HTML
- Forms and Inputs
- CSS — Inline / Internal / External
- Selectors and the Box Model
- Typography, Units, and CSS Variables
- Flexbox
- Grid and Positioning
- Responsive Design and Transitions

HTML — The Structure of the Web

- ▶ HTML = HyperText Markup Language
- ▶ Describes the **structure** of a page, not how it looks
- ▶ An HTML file is just plain text — the browser reads and renders it
- ▶ Every website you've ever visited is HTML at its core

Every Page is HTML

The screenshot shows the Hunter College website (hunter.cuny.edu) with the Chrome DevTools Elements panel open. The website features a navigation bar with links like INFORMATION FOR, QUICK LINKS, DIRECTORY, APPLY, GIVE, and RENT. The main content area has a large image of four students sitting on a bench, with the text "2026 SUMMER SESSION SEIZE THE SUMMER" and a call to action "Find Classes Now". The DevTools panel shows the HTML structure, including the <body> tag with class "home wp-singular page-template page-template-template-full-width page-template-template-full-width-ptp page-page-id-2 wp-theme-codiac tribe-js" and the <div> tag with class "page-wrapper_fullscreen".

hunter.cuny.edu

INFORMATION FOR QUICK LINKS DIRECTORY APPLY GIVE RENT

HUNTER
The City University of New York

≡ MENU 🔍 SEARCH

2026 SUMMER SESSION
SEIZE THE SUMMER

Don't miss out—registration for summer classes is still open! Enjoy greater flexibility while exploring an expanded selection of online courses.

Find Classes Now →

Elements

```
<div>
  <html lang="en" class="fullscreen" data-uw-loader="scroll">
  <head>
  <body class="home wp-singular page-template page-template-template-full-width page-template-template-full-width-ptp page-page-id-2 wp-theme-codiac tribe-js">
    <!-- Google Tag Manager (noscript) -->
    <div class="uw-ony-wayway_p5" data-uw-feature-ignores="true" data-uw-m-ignores="true" title="Accessibility Menu" style="background-color: transparent; important; overflow: initial; important;">
    <div class="uw-s10-bottom-ruler-guide">
    <div class="uw-s10-right-ruler-guide">
    <div class="uw-s10-left-ruler-guide">
    <div class="uw-s10-reading-guide">
    <div class="uw-s12-tooltip" aria-hidden="true">
    <noscript>
    <!-- End Google Tag Manager (noscript) -->
    <!-- MetaFacebook Pixel Code (noscript) -->
    <noscript>
    <!-- End MetaFacebook Pixel Code (noscript) -->
    <header role="banner" class="site-header">
    <main role="main" id="main">
    <div class="page-wrapper_fullscreen">
    <!-- / page-wrapper -->
    <section>
    <main>
    <!-- End main -->
    <footer role="contentinfo">
    <!-- Scroll To Top Button HTML -->
    <button id="w-back-to-top">
    <!-- Begin wp_footer -->
    <script type="speculationrules">
    <script>
    <script type="text/javascript" src="https://s3.amazonaws.com/cuny-content-plugi...>
    <script type="text/javascript" src="https://s3.amazonaws.com/cuny-content-plugi...>
  </div>
  </html>
```

Styles Computed Layout Event Listeners DOM Breakpoints Properties

Filter

div {

height: 100vh;

padding-top: 0;

overflow: hidden;

width: 100%;

display: block;

unicode-bidi: isolate;

Inherited from ...

Tags and Attributes

```
<a href="https://example.com" target="_blank">Visit</a>  
  
  
  
<!-- This is a comment -->
```

- ▶ Tags wrap content: opening `<p>` and closing `</p>`
- ▶ Some are self-closing: `<img>`, `<br>`, `<hr>`
- ▶ **Attributes** add info: href, src, target, width
- ▶ Comments: `<!-- ignored by browser -->`

The Document Skeleton

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My Page</title>
</head>
<body>
  <h1>Hello World</h1>
</body>
</html>
```

- ▶ `<!DOCTYPE html>` — tells browser this is HTML5
- ▶ `<head>` — metadata (not visible on the page)
- ▶ `<body>` — all visible content goes here

Common HTML Tags

Tag	Purpose
<code><h1> – <h6></code>	Headings (h1 biggest, h6 smallest)
<code><p></code>	Paragraph
<code></code>	Link
<code></code>	Image
<code><button></code>	Clickable button
<code><hr></code>	Horizontal rule (divider)
<code>
</code>	Line break

Live Code: snapshot-01

Sourya Saha

Doctoral Student & Computer Science Instructor



I teach Practical Web Development at Hunter College, CUNY. I love building things, breaking them, and explaining how they work.

About

Based in New York. I write code, I teach code, and sometimes I do both at the same time. This portfolio is also a teaching demo for my CSCI 39548 students.

Contact

Reach me at sourya.saha24@login.cuny.edu

[GitHub Profile](#)

[LinkedIn](#)

[Download Resume](#)

Section Items

- HTML Primitives

► **Lists and Tables**

- Semantic HTML
- Forms and Inputs
- CSS — Inline / Internal / External
- Selectors and the Box Model
- Typography, Units, and CSS Variables
- Flexbox
- Grid and Positioning
- Responsive Design and Transitions

Lists and Tables

Structured Content in HTML

Unordered vs Ordered Lists

```
<ul>
  <li>Milk</li>
  <li>Eggs</li>
</ul>

<ol>
  <li>Preheat oven</li>
  <li>Mix ingredients</li>
</ol>
```

- ▶ `<ul>` — bullets, order doesn't matter
- ▶ `<ol>` — numbered, order matters
- ▶ `<li>` — list item (used inside both)

Lists in the Wild

HTML Lists

Unordered list ()

- Apples
- Bananas
- Cherries

Ordered list ()

1. Wake up
2. Make coffee
3. Write code
4. Take a break

Nested list

- Frontend
 - HTML
 - CSS
 - JavaScript
 - React
 - Vue
- Backend
 - Node.js
 - Python

Description list (<dl>)

HTML

The structure of a web page.

CSS

The visual styling of a web page.

JavaScript

The behavior and interactivity of a web page.

Nested Lists

```
<ul>
  <li>Frontend
    <ul>
      <li>HTML</li>
      <li>CSS</li>
      <li>JavaScript</li>
    </ul>
  </li>
  <li>Backend</li>
</ul>
```

- ▶ Lists can nest inside other lists
- ▶ Inner goes inside an

Description Lists

```
<dl>
  <dt>HTML</dt>
  <dd>Structure of the page</dd>
  <dt>CSS</dt>
  <dd>Styling and layout</dd>
</dl>
```

- ▶ `<dl>` — description list
- ▶ `<dt>` — term, `<dd>` — definition
- ▶ Rare but valid — good for glossaries

Table Structure

```
<table>
  <thead>
    <tr>
      <th>Name</th>
      <th>Grade</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Alice</td>
      <td>A</td>
    </tr>
  </tbody>
</table>
```

Tables in the Wild

HTML Tables

Minimal table

Apple	Red
Banana	Yellow
Grape	Purple

Proper table structure

Course	Instructor	Credits	Semester
CSCI 39548 - Practical Web Development	Sourya Saha	3	Summer 2026
CSCI 41500 - Data Communications and Networking	Sourya Saha	3	Spring 2026
CSCI 235 - Software Design	TBD	3	Fall 2026

Merged cells (colspan / rowspan)

Day	Morning	Afternoon
Monday	Office hours	CSCI 39548 lecture (6:40 - 9:48 PM)
Wednesday	Research	
Weekend: catch up on grading		

Spanning Cells

```
<td colspan="2">Spans two columns</td>
```

```
<td rowspan="3">Spans three rows</td>
```

- ▶ `colspan` — merge across columns
- ▶ `rowspan` — merge across rows

Tables are NOT for Layout

- ▶ In the 1990s, developers used `<table>` for page layout
- ▶ That era is over — use Flexbox and Grid (covered later)
- ▶ Tables are **only** for displaying tabular data
- ▶ If your content isn't rows and columns of data, don't use a table

- ▶ Let's open `snapshot-02/lists.html` and `tables.html`
- ▶ Walk through the list types, nesting, and table structure

Section Items

- HTML Primitives

- Lists and Tables

► **Semantic HTML**

- Forms and Inputs

- CSS — Inline / Internal / External

- Selectors and the Box Model

- Typography, Units, and CSS Variables

- Flexbox

- Grid and Positioning

- Responsive Design and Transitions

Semantic HTML

Meaning Over Markup

The Problem with div Everywhere

- ▶ You could build an entire page with only `<div>` tags
- ▶ It would render fine — but no one (including machines) knows what anything means
- ▶ Screen readers can't navigate
- ▶ Search engines can't prioritize
- ▶ Your future self can't read the code

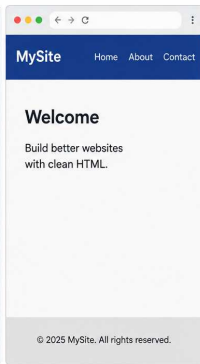
div vs Semantic Tags

Built with `<div>` tags only

HTML

```
<div class="page">
  <div class="header">
    <div class="logo">
      MySite
    </div>
  </div>
  <div class="nav">
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </div>
</div>
<div class="main">
  <div class="section">
    <h1>Welcome</h1>
    <p>Build better websites
      with clean HTML.</p>
  </div>
</div>
<div class="footer">
  <p>© 2025 MySite. All rights
    reserved.</p>
</div>
</div>
```

Rendered in browser

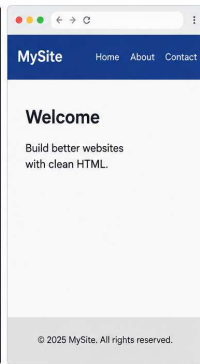


Built with semantic HTML

HTML

```
<header class="header">
  <div class="logo">
    MySite
  </div>
  <nav class="nav">
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </nav>
</header>
<main class="main">
  <section class="section">
    <h1>Welcome</h1>
    <p>Build better websites
      with clean HTML.</p>
  </section>
</main>
<footer class="footer">
  <p>© 2025 MySite. All rights
    reserved.</p>
</footer>
```

Rendered in browser



Same visual result in the browser, but semantic HTML provides meaning, improves accessibility, SEO, and maintainability.

The Semantic Landmark Tags

- ▶ `<header>` — top of page (logo, title, nav)
- ▶ `<nav>` — navigation links
- ▶ `<main>` — primary content (exactly ONE per page)
- ▶ `<section>` — thematic chunk with its own heading
- ▶ `<article>` — self-contained piece (blog post, card)
- ▶ `<aside>` — tangentially related (sidebar, callout)
- ▶ `<footer>` — bottom of page (copyright, links)

Why Semantic Tags Matter: Accessibility

- ▶ Screen readers announce landmarks: “navigation,” “main content,” “footer”
- ▶ Blind users can jump directly to `<main>` or `<nav>`
- ▶ `<div>` is invisible to assistive technology
- ▶ Semantic HTML is the foundation of accessible web development

Why Semantic Tags Matter: SEO

- ▶ Google reads `<main>` first to understand your page's core content
- ▶ `<nav>` helps crawlers find your site's link structure
- ▶ `<article>` signals standalone content worth indexing
- ▶ Better semantics → better search ranking

section vs article

- ▶ `<section>` = a chapter inside a larger whole (can't stand alone)
- ▶ `<article>` = could be lifted out and still make complete sense
- ▶ The test: "Would this make sense in an RSS feed or shared on its own?"
 - ▶ Yes → `<article>`
 - ▶ No → `<section>`

The id Attribute and Anchor Links

```
<nav>
  <a href="#about">About</a>
  <a href="#projects">Projects</a>
</nav>

<section id="about">
  <h2>About Me</h2>
</section>
```

- ▶ id = unique identifier on an element
- ▶ href="#about" jumps to the element with id="about"
- ▶ Also used by CSS and JavaScript selectors

- ▶ Let's open `snapshot-03/index.html`
- ▶ Click the nav links — watch the page jump to sections